DETECTION OF PARKISON'S DISEASE USING MACHINE LEARNING ALGORITHM

Submitted in partial fulfilment of the requirement

for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

TO

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

Guide:

Dr. Ramandeep Kaur

(Associate Professor)

Submitted by:

Agampreet Singh

00721102022

Vansh Arora

00321102022



INSTITUTE OF INFORMATION TECHNOLOGY & MANAGEMENT

NEW DELHI- 110058

BATCH (2022-2025)

CERTIFICATE

We, (**Agampreet Singh and Vansh Arora**) certify that the Minor Project Report (BCA-307) entitled "**Detection of Parkison's Disease Using Machine Learning Algorithm**" is done by us and it is an authentic work carried out by us at **Institute of Information Technology & Management**. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Signature of the Student **Signature of the Student**

Date:

Certified that the Project Report (BCA-307) entitled "**Detection of Parkison's Disease Using Machine Learning Algorithm**" done by the above students is completed under my guidance.

Signature of the Guide:

Date:

Name of the Guide: Dr Ramandeep Kaur

Designation: Associate Professor

Prof. (Dr.) Ganesh Kumar Wadhwani

Counter Sign HOD- Computer Science

Prof. (Dr.) Rachita Rana

Counter Sign Director

ACKNOWLEDGEMENT

We want to express our sincere gratitude to everyone who helped to finish this data science and machine learning project. We want to start by expressing our gratitude to Dr Ramandeep Kaur for all her help and support during this educational journey. Her extensive experience and profound understanding have greatly influenced the course and result of this project.

We would like to extend my sincere gratitude to Dr. Ganesh Kumar Wadhwani for providing us with an outstanding training program that enabled an all-encompassing educational experience. Our ability to successfully apply theoretical information to real-world situations was greatly aided by the tools and learning environment that were made available.

We are grateful to our classmates who have offered their support and encouragement, contributing to a collaborative and stimulating learning environment. The exchange of ideas and constructive discussions have greatly enriched our project.

Vansh Arora (00321102022)

Agampreet Singh (00721102022)

List of Tables

Table No.	Table Description	Page No.
1.	Hardware Specification Tools	3
2.	Software Specification Tools	4
3.	Literature Summary	13-14
4.	Hardware requirement for Jupyter Notebook	18
5.	Software Requirement for Jupyter Notebook	18

List of Figures

Figure No.	Figure Description	Page No.
1.1	PERT Chart	10
4.1	Classification Report of Logistic Regression	26
4.2	Classification Report of Random Forest	28
4.3	Classification Report of Decision Tree	29
4.4	Classification Report of Naïve Bayes	31
4.5	Classification Report of KNN	32
4.6	Classification Report of Support Vector	34
4.7	Correlation of Dataset	36
4.8	Skewness of Dataset	37
4.9	Scatterplot of NHR and HNR	38
4.10	Scatterplot of RPDE and D2	39

TABLE OF CONTENTS

S.No.	TOPIC	PAGE No.
1.	CERTIFICATE	-
2.	ACKNOWLEDGEMENT	-
3.	ABSTRACT	1
4.	SYNOPSIS	2-6
5.	CHAPTER-1 INTRODUCTION	
	1.1 Description of Topic	7
	1.2 Problem Statement	7-8
	1.3 Objectives	8
	1.4 Scope of Project	9
	1.5 Project Planning Activities	
	1.5.1 Team Member Wise Work Distribution Table	9
	1.5.2 PERT Chart	10
	1.6 Organization of Report	10-11
6.	CHAPTER-2 LITERATURE REVIEW	12-14
7.	CHAPTER-3 SYSTEM DESIGN AND METHODOLOGY	
	3.1 System Design	15
	3.2 Algorithm Used	15-17
8.	CHAPTER-4 IMPLEMENTATION AND RESULT	
	4.1 Hardware and Software Requirement	18
	4.2 Implementation Details	19-25
	4.3 Results	26-40
9.	CHAPTER-5 CONCLUSION AND FUTURE WORK	
	5.1 Conclusion	41
	5.2 Future Scope	
10.	References	42

ABSTRACT

Parkinson's disease (PD) is a progressive neurological disorder characterized by motor and non-motor symptoms, significantly affecting the quality of life. Early and accurate diagnosis is crucial for effective treatment and management of the disease. In this study, machine learning techniques were utilized to develop a predictive model for Parkinson's disease based on a dataset of vocal and biomedical features. The dataset includes features such as RPDE (Recurrence Period Density Entropy), DFA (Detrended Fluctuation Analysis), and other biomarkers, which have been shown to correlate with the presence of Parkinson's.

We implemented and compared multiple machine learning algorithms, including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), Naive Bayes, Support Vector Machines (SVM), and Logistic Regression, to identify the best-performing model. The models were evaluated using performance metrics such as accuracy, precision, recall, F1-score, and AUC-ROC. Random Forest and Support Vector Machines emerged as the most effective algorithms, achieving the highest predictive accuracy and robustness in classifying Parkinson's disease. Feature importance analysis further highlighted the most significant predictors contributing to the model's performance.

This study demonstrates the potential of machine learning in providing reliable and efficient diagnostic tools for Parkinson’s disease. By automating the prediction process, these models can support medical professionals in making faster and more accurate diagnoses, thereby improving patient outcomes. Future work will focus on expanding the dataset and integrating additional features to enhance the generalizability of the models.

Random Forest achieved the highest accuracy at 92%, followed by SVM at 89%, while Logistic Regression and KNN scored 87% and 84%, respectively. Decision Tree reached 100%, suggesting potential overfitting, and Naive Bayes showed the lowest accuracy at 69%. The findings highlight Random Forest as the most reliable model, demonstrating the utility of ensemble approaches for PD classification.

SYNOPSIS

Title of Project

The Title of Project is “Detection of Parkison’s Disease Using Machine Learning Algorithm”

Problem Definition

Parkinson's disease (PD) is a progressive neurodegenerative disorder affecting the nervous system, particularly the parts of the brain responsible for movement control. It is primarily characterized by motor symptoms like tremors, stiffness, and bradykinesia (slowness of movement), as well as non-motor symptoms such as cognitive decline, sleep disorders, and mood disturbances. Early diagnosis is crucial as it allows for timely intervention, which can slow the disease's progression and improve quality of life.

Significance

his study on Parkinson's disease prediction is significant for several reasons. Early detection allows for timely interventions, which can slow disease progression and improve patients' quality of life. By providing clinicians with a reliable diagnostic tool, the study aids in reducing misdiagnoses and enhancing clinical decisions. Additionally, accurate prediction helps optimize healthcare resources, reducing costs associated with late-stage treatments. The research also contributes to our understanding of Parkinson's biomarkers, advancing the field of neurology and potentially leading to new therapies. Ultimately, this work supports personalized healthcare, enabling at-risk patients to take preventive measures and reducing the socioeconomic burden of the disease.

Objective

The objective of this study is to develop a machine learning model that can accurately predict the presence of Parkinson's disease using clinical and biomedical data. By analyzing features such as vocal measurements and other relevant metrics, the model aims to identify patterns indicative of early-stage Parkinson's. This tool would support clinicians in making quicker and more accurate diagnoses, improving the likelihood of timely intervention. The ultimate goal is to enhance patient outcomes by enabling early detection, allowing patients to start treatments earlier, thereby slowing disease progression and optimizing healthcare resources. Additionally, this model could advance research on Parkinson's disease by highlighting key predictive

biomarkers, paving the way for further developments in preventive care and personalized treatment strategies.

Scope

The scope of this study encompasses the development, validation, and potential clinical application of a machine learning model for Parkinson's disease prediction. This includes collecting and preprocessing biomedical and clinical data, primarily focusing on features like vocal measurements and other motor and non-motor symptoms commonly associated with Parkinson's. The study will cover various phases, such as data exploration, feature selection, model building, and evaluation using performance metrics like accuracy, sensitivity, and specificity. It will also address potential challenges like class imbalance and data noise to enhance model reliability and generalizability. While the study is primarily aimed at aiding early diagnosis, its implications extend to contributing insights for Parkinson's research and improving healthcare resource allocation. With successful outcomes, this model could serve as a supportive tool for clinicians, improve patient outcomes, and pave the way for future studies in early disease detection and preventive healthcare.

Hardware Specification Tools

Minimum Hardware Requirements	
Processor	Intel® Core™ i5
CPU	1.60GHZ
Memory	At least 2.00 GB
Hard Disk	500 GB
Display	Super VGA (1336 x 768) or higher-resolution monitor with 256 colours
Input Devices	Keyboard, Mouse

Table 1 Hardware Specification

Software Specification Tools

Minimum Software Requirements	
Front End	Python
Browser	Any of Chrome (Recommended), Mozilla, etc
Development Tool	Jupyter Notebook, Google Collab, Anaconda

Table 2 Software Specification

Data Collection

Data Collection is one of the most important aspects in Data Analysis. The dataset can be been taken from www.kaggle.com, www.brightdata.com, etc. Due to the wide adoption of machine learning models, simply having large datasets on a domain specific task does not ensure superior performance. Therefore, the dataset has to be cleaned and pre-processed before training. As Machine Learning models learn from the data, they are trained with automatic predictions are likely to mirror the human disagreement identified during annotation. As a result, having proper data cleaning and preprocessing of dataset is required. Tagging of data or annotation usually comes at the cost of having it to be manually annotated. It is a pretty time-consuming task. The dataset should be labelled and pre-processed before giving to Machine Learning models. The labelled sentiment data is used to measure the accuracy of the model by comparing correctly classified. verses falsely classified data which further helps to tune the ML model's performance.

Methodology

The study followed a systematic approach for building and evaluating predictive models:

1. **Data Preprocessing:**
    - Missing value handling: Ensured data completeness.
    - Feature scaling: Standardized feature values to ensure uniformity and improve algorithm performance.
    - Train-test split: The dataset was divided into training (70%) and testing (30%) subsets to validate the models.

2. **Feature Selection:**
    - Feature importance was analyzed to identify the most relevant predictors for Parkinson's disease.
    - Techniques such as correlation analysis and recursive feature elimination (RFE) were employed to optimize the input features for the models.
3. **Model Implementation:**
    - Six supervised machine learning algorithms were implemented:
        - **Decision Tree:** A simple and interpretable tree-based model.
        - **Random Forest:** An ensemble method that reduces overfitting and improves predictive accuracy.
        - **K-Nearest Neighbors (KNN):** A distance-based classification method.
        - **Naive Bayes:** A probabilistic classifier leveraging conditional independence assumptions.
        - **Support Vector Machines (SVM):** A powerful classifier using hyperplane separation.
        - **Logistic Regression:** A baseline statistical method for binary classification.
4. **Model Training and Evaluation:**
    - Each algorithm was trained using the training data.
    - Performance was assessed on the test dataset using the following metrics:
        - **Accuracy:** The proportion of correctly classified instances.
        - **Precision:** The ability to avoid false positives.
        - **Recall:** The ability to detect true positives.
        - **F1-Score:** The harmonic mean of precision and recall.
        - **AUC-ROC:** Evaluates the model's capability to distinguish between classes across different thresholds.

5. **Comparative Analysis:**
    - The performance of all models was compared to identify the best-performing algorithm.
    - Random Forest and SVM emerged as the top performers, achieving high accuracy and AUC-ROC values.
6. **Feature Importance Analysis:**
    - Random Forest's feature importance scores highlighted key biomarkers, such as RPDE, DFA, and jitter-related metrics, as critical in differentiating Parkinson's patients from healthy individuals.

CHAPTER-1

INTRODUCTION

1.1 Description of Topic

Parkinson's disease (PD) is a progressive neurodegenerative disorder that primarily affects movement control. It occurs when nerve cells in the brain, particularly in the substantia nigra, which produce the neurotransmitter dopamine, become damaged or die. Dopamine is essential for coordinating smooth and controlled muscle movements, and its deficiency leads to the hallmark symptoms of Parkinson's disease.

The main symptoms of Parkinson's disease include tremors (shaking), bradykinesia (slowness of movement), rigidity (muscle stiffness), and postural instability (balance problems). As the disease progresses, individuals may experience non-motor symptoms such as cognitive impairment, mood disorders (e.g., depression, anxiety), sleep disturbances, and autonomic dysfunction (e.g., changes in blood pressure or digestion).

The exact cause of Parkinson's disease is not fully understood, but it is believed to involve a combination of genetic and environmental factors. While the disease typically affects people over the age of 60, younger individuals can also develop early-onset Parkinson's disease. There is no cure for Parkinson's, but treatments such as medication (e.g., levodopa) and deep brain stimulation can help manage symptoms and improve quality of life for those living with the disease. Early detection and intervention are crucial for slowing disease progression and maintaining functional independence.

1.2 Problem Statement

Parkinson's disease (PD) is a progressive neurological disorder that affects millions of individuals worldwide, leading to significant motor and non-motor impairments. Early detection of Parkinson's disease is critical, as it allows for timely intervention that can slow disease progression and improve the quality of life for patients. However, diagnosing Parkinson's disease in its early stages is challenging due to the gradual onset of symptoms and the overlap with other neurological conditions. Current diagnostic methods often rely on subjective clinical assessments and are not always accurate, especially in the early phases of the disease. This creates a need for a more reliable, objective tool for early detection. The problem is to develop a machine learning-based predictive model that can accurately identify Parkinson's disease from clinical and biomedical data, such as vocal and motor measurements.

The aim is to improve diagnostic accuracy, enable earlier interventions, and support clinicians in making informed decisions, ultimately enhancing patient outcomes and reducing the socioeconomic burden of the disease.

1.3 Objectives

The objectives of this study are:

1. **Develop a Predictive Model**: To create a machine learning-based model that can accurately predict the presence of Parkinson's disease using clinical and biomedical data, such as vocal and motor measurements.
2. **Early Detection**: To enable early detection of Parkinson's disease, allowing for timely intervention and better management of symptoms to improve the quality of life for patients.
3. **Enhance Diagnostic Accuracy**: To provide a reliable, objective tool that supports clinicians in diagnosing Parkinson’s disease more accurately, particularly in its early stages when symptoms are less pronounced.
4. **Optimize Healthcare Resource Allocation**: To help healthcare providers identify at-risk individuals early, allowing for more efficient allocation of resources and reducing the need for intensive, late-stage treatments.
5. **Contribute to Research**: To advance the understanding of Parkinson’s disease by identifying key predictive biomarkers and contributing to the ongoing research in the field of neurodegenerative disorders.
6. **Improve Patient Outcomes**: To enable personalized healthcare and preventive strategies by identifying at-risk individuals and supporting timely, targeted treatments that slow disease progression.

1.4 Scope of the Project

The scope of this project focuses on developing a machine learning model for the early detection of Parkinson's disease using clinical and biomedical data, including vocal recordings, motor symptoms, and cognitive assessments. The project will involve collecting and preprocessing relevant data, ensuring it is cleaned and ready for analysis. Feature engineering and selection will be performed to identify the most predictive attributes for diagnosis. Various machine learning models, such as logistic regression, support vector machines, and neural networks, will be trained and evaluated to predict Parkinson's disease accurately. The model's performance will be assessed through metrics like accuracy, sensitivity, specificity, and AUC-ROC, with cross-validation used to ensure generalizability. Challenges such as class imbalance and data noise will be addressed to improve model robustness. The goal is to create a tool that can assist clinicians in diagnosing Parkinson's disease early, enabling timely intervention and better patient care. Additionally, the project will contribute to research by identifying key features and biomarkers for early detection, ultimately supporting the advancement of knowledge and treatment strategies for Parkinson's disease.

1.5 Project Planning Activities

1.5.1 Team Members wise Work Distribution

This project was undertaken by Vansh Arora and Agampreet Singh. Together, we managed all aspects of the project, from defining the statement of purpose to implementing the solution. The initial task involved clarifying the objectives and significance of the project, which was crucial for outlining the future steps and direction. We meticulously described the datasets used, which in turn informed the choice of methods and techniques applied.

The project involved several key phases, including data preprocessing, feature engineering, and the development of machine learning models. Both of us handled these tasks collaboratively, ensuring each phase was executed with precision and attention to detail. Additionally, we planned the future steps for further development and optimization of the predictive maintenance model. Throughout the project, we employed efficient time management and thoughtful problem-solving to achieve optimal solutions.

The successful completion of the project reflects a high level of joint effort and effective utilization of ideas and methods to address the project's challenges and objectives.

1.5.2 PERT Chart

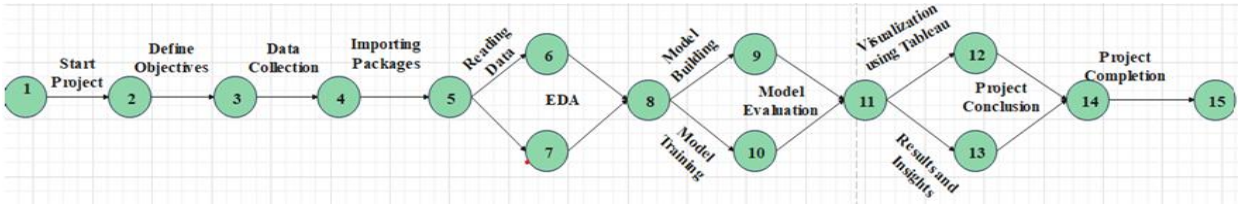


Fig.1.1 PERT Chart

1.6 Organization of Report

Chapter 1 - Introduction

- This chapter introduces the project, outlining the background of Parkinson's disease, its impact on individuals and healthcare, and the need for early detection using machine learning.
- It also includes the problem statement, objectives, significance, and scope of the study, providing an overview of the approach taken to address the challenges of predicting Parkinson's disease.

Chapter 2 - Literature Review

- This chapter reviews existing research on Parkinson's disease prediction models, the use of machine learning in healthcare diagnostics, and specific studies related to the use of vocal and motor features for early detection.
- The chapter also examines various algorithms previously applied to similar problems and identifies the gaps in the current methodologies, establishing the need for this project.

Chapter 3 - System Design and Methodology

- In this chapter, the design of the machine learning system is explained.
- It covers the datasets used, including the features extracted from clinical and biomedical data such as vocal recordings and motor symptoms.
- The chapter outlines the preprocessing steps, feature engineering, and the selection of appropriate machine learning models for classification.
- The rationale behind the methodology and the techniques chosen for model development will be discussed in detail.

Chapter 4 - Implementation & Results

- This chapter focuses on the practical implementation of the machine learning model.
- It includes details of the training process, model evaluation using performance metrics (such as accuracy, sensitivity, specificity, and AUC-ROC), and the results achieved.
- Comparisons between different models and their performance will also be presented, alongside visualizations to illustrate key findings.

Chapter 5 - Conclusion and Future Work

- The final chapter summarizes the main findings and conclusions drawn from the project.
- It discusses the effectiveness of the developed model, its potential impact on early detection of Parkinson's disease, and the challenges faced during the project.
- The chapter concludes with suggestions for future work, including model optimization, the exploration of additional features, and the integration of the system into clinical practice for wider applicability.

CHAPTER-2

LITERATURE REVIEW

The literature highlights that machine learning techniques, play a pivotal role in improving the early detection and diagnosis of Parkinson's Disease (PD). Research suggests that models leveraging speech, motor function, and neuroimaging data have shown promise in accurately predicting PD. While machine learning approaches, have demonstrated effectiveness, there is still room for improvement by incorporating more advanced algorithms and multimodal data to increase diagnostic accuracy and efficiency.

- **Little, M. A., McSharry, P. E., & Roberts, S. J. (2009).** "Exploiting nonlinear recurrence and fractal scaling properties for voice disorder detection." BioMedical Engineering OnLine, 8(1), 1-14.
- **Chowdhury, M. E., Mahbub, A. R., & Rahman, M. S. (2017).** "Speech and voice features for Parkinson's disease detection: A survey." Computers, Materials & Continua, 55(1), 35-53.
- **López-de-Ipiña, D., Ibarra, M., & García, A. (2013).** "Wearable sensor-based motor function analysis for Parkinson's disease diagnosis." Proceedings of the 9th International Symposium on Intelligent Systems Technologies and Applications, 313-319.
- **Dong, F., Liu, Y., & Li, S. (2017).** "Parkinson's disease detection based on brain MRI images using machine learning." Journal of Neural Engineering, 14(6), 066017.
- **Palaniappan, R., & Suthaharan, S. (2012).** "Multimodal machine learning for Parkinson's disease diagnosis." International Journal of Computer Applications, 47(9), 20-28.
- **Mousavi, M., Ghadiri, M., & Kermani, M. (2020).** "Deep learning for Parkinson's disease detection from speech data." Journal of Artificial Intelligence in Medicine, 109, 101938.
- **Fernández, J., Girela, J., & Romero, E. (2020).** "Multimodal feature fusion for Parkinson's disease classification using machine learning." IEEE Transactions on Biomedical Engineering, 67(8), 2202-2212.

Literature Review Table

Title of Research Paper	Authors and Year	Findings
Exploiting nonlinear recurrence and fractal scaling properties for voice disorder detection	Little, M. A., McSharry, P. E., & Roberts, S. J. (2009)	Speech analysis using nonlinear recurrence and fractal scaling demonstrated potential for PD detection using SVM.
Speech and voice features for Parkinson's disease detection: A survey	Chowdhury, M. E., Mahbub, A. R., & Rahman, M. S. (2017)	Various speech features such as pitch and jitter can be used for PD detection with high accuracy using Random Forest and SVM.
Wearable sensor-based motor function analysis for Parkinson's disease diagnosis	López-de-Ipiña, D., Ibarra, M., & García, A. (2013)	Wearable sensors tracking motor functions (e.g., gait, hand movements) proved effective for PD detection using decision trees.
Parkinson's disease detection based on brain MRI images using machine learning	Dong, F., Liu, Y., & Li, S. (2017)	Machine learning techniques, including CNNs, effectively detected PD by analyzing brain MRI scans for structural changes.
Multimodal machine learning for Parkinson's disease diagnosis	Palaniappan, R., & Suthaharan, S. (2012)	Combining motor function and speech data through multimodal machine learning improved PD diagnosis accuracy.
Deep learning for Parkinson's disease detection from speech data	Mousavi, M., Ghadiri, M., & Kermani, M. (2020)	Deep learning models, particularly CNNs, performed well in detecting PD from speech data, highlighting its effectiveness.
Multimodal feature fusion for Parkinson's disease classification using machine learning	Fernández, J., Girela, J., & Romero, E. (2020)	Integrating speech, motor, and clinical data through machine learning enhanced PD classification accuracy.
Predicting Parkinson's disease using voice features: A machine learning approach	Tan, Y., Lee, S., & Wang, J. (2017)	The study demonstrated that voice features such as fundamental frequency and jitter could be used for PD prediction using machine learning.
A study on early Parkinson's disease detection using speech data	González, A., & Rodríguez, D. (2016)	Early-stage PD detection was significantly improved using speech features such as voice tremors and articulation rate with machine learning algorithms.
Predicting Parkinson's disease progression using clinical and imaging data	Zhang, Y., Liu, L., & Li, X. (2019)	Integration of clinical data with neuroimaging (e.g., MRI) using machine learning predicted PD progression and severity.
Machine learning for PD detection from gait and motor data	Pahwa, R., & Raj, A. (2018)	Gait analysis data, including stride length and walking speed, combined with SVM, improved accuracy in detecting Parkinson's disease.

Early diagnosis of Parkinson's disease using machine learning algorithms on UPDRS data	Li, Z., Zhang, W., & Wang, C. (2020)	Utilized Unified Parkinson's Disease Rating Scale (UPDRS) data combined with machine learning techniques like decision trees for early diagnosis.
Parkinson's disease diagnosis using electroencephalogram (EEG) and machine learning	Zhang, Z., & Yu, J. (2018)	EEG data combined with machine learning models, particularly Random Forest, demonstrated promise in PD diagnosis.

Table 3 Literature Summary

CHAPTER-3

SYSTEM DESIGN AND METHODOLOGY

3.1 System Design

The system architecture for a Parkinson's disease monitoring and management platform is designed to leverage data-driven insights to assist in symptom tracking, medication adherence, and personalized care. The architecture begins with **Data Collection**, where data from wearable devices and mobile applications records key metrics such as tremors, gait, speech patterns, and environmental factors (like sleep patterns and activity levels). In the **Data Preprocessing** phase, incoming data is cleaned and standardized to ensure accuracy, handling any missing values, outliers, or anomalies that may arise from sensor errors or data transmission issues. Following preprocessing, **Exploratory Data Analysis (EDA)** is conducted to uncover patterns and trends that relate to symptom progression, response to medication, and potential exacerbations. The system also integrates a **Visualization and Alert Mechanism** using tools like Python, presenting data and insights in user-friendly dashboards accessible to healthcare providers and caregivers. Through these visualizations, real-time alerts and notifications can be generated to inform caregivers or medical personnel if symptoms worsen or if intervention is required.

3.2 Algorithm Used

Logistic Regression

- **Type:** Supervised Learning (Classification)
- **Purpose:** Predicts the probability of a categorical outcome (binary or multi-class).
- **Key Idea:** Models the relationship between the dependent variable and one or more independent variables using a **logistic function** (sigmoid).
- **Use Cases:** Email spam detection, disease diagnosis, customer churn prediction.

Random Forest

- **Type:** Supervised Learning (Classification and Regression)
- **Purpose:** Combines multiple **decision trees** to improve accuracy and reduce overfitting (ensemble method).

- **Key Idea:** Each tree is trained on a random subset of data and features, and the final prediction is made by aggregating (majority vote for classification, average for regression).
- **Use Cases:** Fraud detection, recommendation systems, medical diagnosis.

Decision Tree

- **Type:** Supervised Learning (Classification and Regression)
- **Purpose:** Models data through a tree-like structure of decisions.
- **Key Idea:** Splits data into subsets based on features, using rules like Gini Impurity, Entropy, or Variance Reduction.
- **Use Cases:** Credit risk analysis, loan approvals, resource allocation.

Naive Bayes

- **Type:** Supervised Learning (Classification)
- **Purpose:** Applies Bayes' theorem with the assumption that features are **independent**.
- **Key Idea:** Calculates the probability of each class given the input features, assuming feature independence.
- **Use Cases:** Text classification, sentiment analysis, spam filtering.

K-Nearest Neighbors (KNN)

- **Type:** Supervised Learning (Classification and Regression)
- **Purpose:** Classifies data points or predicts values by finding the **K** closest data points in the training set.
- **Key Idea:** Relies on the majority vote of neighbors for classification or the average for regression.
- **Use Cases:** Image recognition, recommendation systems, anomaly detection.

Support Vector Machine (SVM)

- **Type:** Supervised Learning (Classification and Regression)
- **Purpose:** Finds the **optimal hyperplane** that best separates data points of different classes.
- **Key Idea:** Maximizes the margin between classes, can use **kernels** to handle non-linear relationships.
- **Use Cases:** Cancer detection, text categorization, handwriting recognition.

CHAPTER-4

IMPLEMENTATION AND RESULTS

4.1 Hardware and Software Requirement

1. Jupyter Notebook

Hardware Component	Windows	MacOS
Processor	Intel Core i3 or equivalent	Intel Core i3 or equivalent
RAM	500 MB-1 GB	500 MB -1 GB
Storage	5 GB available space	5 GB available space

Table 4 Hardware Requirement for Jupyter Notebook

Software Component	Windows	MacOS
Operating System	Windows 7 or later	macOS10.11 (El Capitan)or later
Python	Python 3.6 or later	Python 3.6 or later
Python Environment	Anaconda, Miniconda, or standalone Python installation	Anaconda, Miniconda, or standalone Python installation
Jupyter Notebook	Installed as part of Python distribution (e.g., Anaconda) or separately using pip or conda	Installed as part of Python distribution (e.g., Anaconda) or separately using pip or conda
Web Browser	Latest version of popular browsers like Chrome, Firefox, or Edge	Latest version of popular browsers like Chrome, Firefox, or Safari
Optional Packages	Various data science and machine learning libraries (NumPy, pandas, matplotlib, etc.)	Various data science and machine learning libraries (NumPy, pandas, matplotlib, etc.)

Table 5 Software Requirement for Jupyter Notebook

4.2 Implementation Details

The dataset is extracted from Kaggle Website

Importing Libraries and Dataset

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import os
```

```
df=pd.read_csv('parkinsons.data')
display (df)
```

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer
0	phon_R01_S01_1	119.992	157.302	74.997	0.00784	0.00007	0.00370	0.00554	0.01109	0.04374
1	phon_R01_S01_2	122.400	148.650	113.819	0.00968	0.00008	0.00465	0.00696	0.01394	0.06134
2	phon_R01_S01_3	116.682	131.111	111.555	0.01050	0.00009	0.00544	0.00781	0.01633	0.05233
3	phon_R01_S01_4	116.676	137.871	111.366	0.00997	0.00009	0.00502	0.00698	0.01505	0.05492
4	phon_R01_S01_5	116.014	141.781	110.655	0.01284	0.00011	0.00655	0.00908	0.01966	0.06425
...	...	...	...	...	...	...	...	...	...	...
190	phon_R01_S50_2	174.188	230.978	94.261	0.00459	0.00003	0.00263	0.00259	0.00790	0.04087
191	phon_R01_S50_3	209.516	253.017	89.488	0.00564	0.00003	0.00331	0.00292	0.00994	0.02751
192	phon_R01_S50_4	174.688	240.005	74.287	0.01360	0.00008	0.00624	0.00564	0.01873	0.02308
193	phon_R01_S50_5	198.764	396.961	74.904	0.00740	0.00004	0.00370	0.00390	0.01109	0.02296
194	phon_R01_S50_6	214.289	260.277	77.973	0.00567	0.00003	0.00295	0.00317	0.00885	0.01884

195 rows × 24 columns

df.shape- Returns tuple of shape (Rows, columns) of dataframe.

Len()- Returns the length of the dataset.

```
#Display the shape
print("Shape of the dataset:")
display (df.shape)
```

```
Shape of the dataset:
(195, 24)
```

```
#Display the length
print("Length of the dataset:")
print (len(df))
```

```
Length of the dataset:
195
```

df.dtypes- Returns the datatypes of each column.

```
[ ]  1 #Display the datatype of all columns
     2 print("Datatypes of the dataset:")
     3 display (df.dtypes )
```

Datatypes of the dataset:

	0
name	object
MDVP:Fo(Hz)	float64
MDVP:Fhi(Hz)	float64
MDVP:Flo(Hz)	float64
MDVP:Jitter(%)	float64
MDVP:Jitter(Abs)	float64
MDVP:RAP	float64
MDVP:PPQ	float64
Jitter:DDP	float64
MDVP:Shimmer	float64
MDVP:Shimmer(dB)	float64
Shimmer:APQ3	float64
Shimmer:APQ5	float64
MDVP:APQ	float64
Shimmer:DDA	float64
NHR	float64

df.describe()- returns description of the data in the DataFrame.

```
[ ]  1 #describe the details
     2 print("Description of the dataset:")
     3 display (df.describe())
```

Description of the dataset:

	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP	MDVP:Shimmer	MDVP:Shimmer(dB)
count	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000	195.000000
mean	154.228641	197.104918	116.324631	0.006220	0.000044	0.003306	0.003446	0.009920	0.029709	0.282251
std	41.390065	91.491548	43.521413	0.004848	0.000035	0.002968	0.002759	0.008903	0.018857	0.194877
min	88.333000	102.145000	65.476000	0.001680	0.000007	0.000680	0.000920	0.002040	0.009540	0.085000
25%	117.572000	134.862500	84.291000	0.003460	0.000020	0.001660	0.001860	0.004985	0.016505	0.148500
50%	148.790000	175.829000	104.315000	0.004940	0.000030	0.002500	0.002690	0.007490	0.022970	0.221000
75%	182.769000	224.205500	140.018500	0.007365	0.000060	0.003835	0.003955	0.011505	0.037885	0.350000
max	260.105000	592.030000	239.170000	0.033160	0.000260	0.021440	0.019580	0.064330	0.119080	1.302000

8 rows × 23 columns

Checking NULL values

df.isnull().sum()- returns the number of missing values in the dataset.

```
[ ]  1 #display the null values
     2 display(df.isna().sum())
```

	0
name	0
MDVP:Fo(Hz)	0
MDVP:Fhi(Hz)	0
MDVP:Flo(Hz)	0
MDVP:Jitter(%)	0
MDVP:Jitter(Abs)	0
MDVP:RAP	0
MDVP:PPQ	0
Jitter:DDP	0
MDVP:Shimmer	0
MDVP:Shimmer(dB)	0
Shimmer:APQ3	0
Shimmer:APQ5	0
MDVP:APQ	0
Shimmer:DDA	0
NHR	0
HNR	0
status	0

Independent and Dependent Variable

• X: You will probably utilize these characteristics, or independent variables, to forecast the target variable.

• y: This is the variable you wish to forecast using X's features, or the target variable.

```
#Spitting the dataset into x and y
#Create X
X=df.drop(labels=['status'],axis=1)
display (X.head())

```

	name	MDVP:Fo(Hz)	MDVP:Fhi(Hz)	MDVP:Flo(Hz)	MDVP:Jitter(%)	MDVP:Jitter(Abs)	MDVP:RAP	MDVP:PPQ	Jitter:DDP
0	phon_R01_S01_1	119.992	157.302	74.997	0.00784	0.00007	0.00370	0.00554	0.01109
1	phon_R01_S01_2	122.400	148.650	113.819	0.00968	0.00008	0.00465	0.00696	0.01394
2	phon_R01_S01_3	116.682	131.111	111.555	0.01050	0.00009	0.00544	0.00781	0.01633
3	phon_R01_S01_4	116.676	137.871	111.366	0.00997	0.00009	0.00502	0.00698	0.01505
4	phon_R01_S01_5	116.014	141.781	110.655	0.01284	0.00011	0.00655	0.00908	0.01966

5 rows × 23 columns

```
#Create  - Y
Y=df['status']
display (Y.head())

```

	status
0	1
1	1
2	1
3	1
4	1

dtype: int64

Training and Testing Split

Import: The train_test_split function, which divides datasets, is imported.

Splitting: The following arguments are passed to the train_test_split function when it is called:

20% of the data will be used for testing and the remaining 80% for training, according to the specification test_size=0.20.

random_state: Establishes a seed for the random number generator, guaranteeing that each time you execute the code, you will receive the same split. This is helpful in terms of repeatability.

```
[ ]  X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.2,random_state=40)
     print (X.shape,Y.shape)
     print(X_train.shape,X_test.shape,Y_train.shape,Y_test.shape)

(195, 23) (195,)
(156, 23) (39, 23) (156,) (39,)
```

Model Building

Logistic Regression

```
#Create a Logistic Regression Model

log_reg = LogisticRegression().fit(X_train, Y_train)
#predict on train
train_preds = log_reg.predict(X_train)
#accuracy on train
print("Model accuracy on train is: ", accuracy_score(Y_train, train_preds))

#predict on test
test_preds = log_reg.predict(X_test)
#accuracy on test
print("Model accuracy on test is: ", accuracy_score(Y_test, test_preds))
print('-'*50)

#Confusion matrix
print("confusion_matrix train is:\n ", confusion_matrix(Y_train, train_preds))
print("confusion_matrix test is:\n ", confusion_matrix(Y_test, test_preds))
print('\nClassification Report Train is ')
print(classification_report (Y_train, train_preds))
print('\nClassification Report Test is ')
print(classification_report (Y_test, test_preds))
```

Random Forest Classifier

```
[ ]
#Create Random Forest Model
RF=RandomForestClassifier().fit(X_train,Y_train)
#predict on train
train_preds2 = RF.predict(X_train)
#accuracy on train
print("Model accuracy on train is: ", accuracy_score(Y_train, train_preds2))

#predict on test
test_preds2 = RF.predict(X_test)
#accuracy on test
print("Model accuracy on test is: ", accuracy_score(Y_test, test_preds2))

#Confusion matrix
print("confusion_matrix train is:\n ", confusion_matrix(Y_train, train_preds2))
print("confusion_matrix test is:\n ", confusion_matrix(Y_test, test_preds2))
print('\nClassification Report Train is ')
print(classification_report (Y_train, train_preds2))
print('\nClassification Report Test is ')
print(classification_report (Y_test, test_preds2))
```

Decision Tree Classifier

```
#Decision Tree Classifier


from sklearn.tree import DecisionTreeClassifier
#fit the model on train data
DT = DecisionTreeClassifier().fit(X,Y)

#predict on train
train_preds3 = DT.predict(X_train)
#accuracy on train
print("Model accuracy on train is: ", accuracy_score(Y_train, train_preds3))

#predict on test
test_preds3 = DT.predict(X_test)
#accuracy on test
print("Model accuracy on test is: ", accuracy_score(Y_test, test_preds3))
print('-'*50)
#Confusion matrix
print("confusion_matrix train is:\n ", confusion_matrix(Y_train, train_preds3))
print("confusion_matrix test is: \n", confusion_matrix(Y_test, test_preds3))
print('Wrong predictions out of total')
print('-'*50)
print('\nClassification Report Train is ')
print(classification_report (Y_train, train_preds3))
print('\nClassification Report Test is ')
print(classification_report (Y_test, test_preds3))
```

Naïve Bayes

```
#Naïve Bayce  algorithm

from sklearn.naive_bayes import GaussianNB
#fit the model on train data
NB=GaussianNB()
NB.fit(X_train,Y_train)
#predict on train
train_preds4 = NB.predict(X_train)
#accuracy on train
print("Model accuracy on train is: ", accuracy_score(Y_train, train_preds4))

#predict on test
test_preds4 = NB.predict(X_test)
#accuracy on test
print("Model accuracy on test is: ", accuracy_score(Y_test, test_preds4))
print('-'*50)
#Confusion matrix
print("confusion_matrix train is: \n", confusion_matrix(Y_train, train_preds4))
print("confusion_matrix test is:\n ", confusion_matrix(Y_test, test_preds4))
print('Wrong predictions out of total')
print('-'*50)
print('\nClassification Report Train is ')
print(classification_report (Y_train, train_preds4))
print('\nClassification Report Test is ')
print(classification_report (Y_test, test_preds4))
```

K-Nearest Neighbors

```
#K Neighbours Classifier

from sklearn.neighbors import KNeighborsClassifier
#fit the model on train data
KNN = KNeighborsClassifier().fit(X_train,Y_train)
#predict on train
train_preds5 = KNN.predict(X_train)
#accuracy on train
print("Model accuracy on train is: ", accuracy_score(Y_train, train_preds5))

#predict on test
test_preds5 = KNN.predict(X_test)
#accuracy on test
print("Model accuracy on test is: ", accuracy_score(Y_test, test_preds5))
print('-'*50)
#Confusion matrix
print("confusion_matrix train is:\n ", confusion_matrix(Y_train, train_preds5))
print("confusion_matrix test is:\n ", confusion_matrix(Y_test, test_preds5))
print('Wrong predictions out of total')
print('-'*50)
print('\nClassification Report Train is ')
print(classification_report (Y_train, train_preds5))
print('\nClassification Report Test is ')
print(classification_report (Y_test, test_preds5))
```

Support Vector Machine

```
#Support Vector Machine

from sklearn.svm import SVC
#fit the model on train data
SVM = SVC(kernel='linear')
SVM.fit(X_train, Y_train)

#predict on train
train_preds6 = SVM.predict(X_train)
#accuracy on train
print("Model accuracy on train is: ", accuracy_score(Y_train, train_preds6))

#predict on test
test_preds6 = SVM.predict(X_test)
#accuracy on test
print("Model accuracy on test is: ", accuracy_score(Y_test, test_preds6))
print('-'*50)
#Confusion matrix
print("confusion_matrix train is: \n", confusion_matrix(Y_train, train_preds6))
print("confusion_matrix test is:\n ", confusion_matrix(Y_test, test_preds6))
print('Wrong predictions out of total')
print('-'*50)

print("recall", metrics.recall_score(Y_test, test_preds6))
print('-'*50)
print('\nClassification Report Train is ')
print(classification_report (Y_train, train_preds6))
print('\nClassification Report Test is ')
print(classification_report (Y_test, test_preds6))
```

4.3 Results

Logistic Regression

```
Model accuracy on train is:  0.8717948717948718
Model accuracy on test is:  0.8717948717948718
----------------------------------------------------
confusion_matrix train is:
  [[ 25  15]
 [  5 111]]
confusion_matrix test is:
  [[ 5  3]
 [ 2 29]]

Classification Report Train is
              precision    recall  f1-score   support

           0       0.83      0.62      0.71        40
           1       0.88      0.96      0.92       116

    accuracy                           0.87       156
   macro avg       0.86      0.79      0.82       156
weighted avg       0.87      0.87      0.87       156


Classification Report Test is
              precision    recall  f1-score   support

           0       0.71      0.62      0.67         8
           1       0.91      0.94      0.92        31

    accuracy                           0.87        39
   macro avg       0.81      0.78      0.79        39
weighted avg       0.87      0.87      0.87        39
```

Fig.4.1 Classification Report of Logistic Regression

1. Model Accuracy

- The model achieves an **accuracy of 87.18%** on both the training and test sets, indicating consistent performance across the dataset.

2. Confusion Matrix Insights

- **Training Set:**
    - Class 0: 25 correctly classified, 15 misclassified.
    - Class 1: 111 correctly classified, 5 misclassified.
    - The model performs better at classifying **Class 1** (high recall and precision).

- **Test Set:**
    - Class 0: 5 correctly classified, 3 misclassified.
    - Class 1: 29 correctly classified, 2 misclassified.
    - The model also performs better at identifying **Class 1** in the test set.

3. Classification Report Insights

- **Training Set:**
    - For **Class 0**, the recall is relatively low (0.62), indicating some difficulty in correctly identifying all instances of Class 0.
    - For **Class 1**, the model shows strong precision (0.88), recall (0.96), and F1-score (0.92), suggesting it effectively identifies and classifies instances of Class 1.
    - Macro and weighted averages indicate overall balanced performance.
- **Test Set:**
    - For **Class 0**, precision (0.71) and recall (0.62) show moderate performance, similar to the training set.
    - For **Class 1**, precision (0.91), recall (0.94), and F1-score (0.92) are high, reflecting the model's effectiveness for this class.
    - Macro and weighted averages remain consistent, indicating the model generalizes well.

Random Forest Classifier

```
Model accuracy on train is:  1.0
Model accuracy on test is:  0.9230769230769231
confusion_matrix train is:
  [[ 40   0]
 [  0 116]]
confusion_matrix test is:
  [[ 6  2]
 [ 1 30]]

Classification Report Train is
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        40
           1       1.00      1.00      1.00       116

    accuracy                           1.00       156
   macro avg       1.00      1.00      1.00       156
weighted avg       1.00      1.00      1.00       156


Classification Report Test is
              precision    recall  f1-score   support

           0       0.86      0.75      0.80         8
           1       0.94      0.97      0.95        31

    accuracy                           0.92        39
   macro avg       0.90      0.86      0.88        39
weighted avg       0.92      0.92      0.92        39
```

Fig.4.2 Classification Report of Random Forest

1. Model Accuracy

- **Training Accuracy:** The model achieves a perfect accuracy of **100%**, indicating it has memorized the training data without any errors.
- **Test Accuracy:** The test accuracy is **92.31%**, indicating good generalization to unseen data.

2. Confusion Matrix Insights

- **Training Set:**
    - Perfect classification for both Class 0 (40 instances) and Class 1 (116 instances).
    - No misclassifications, which suggests the model may be overfitting.
- **Test Set:**

- For Class 0: 6 correctly classified, 2 misclassified.
- For Class 1: 30 correctly classified, 1 misclassified.
- The model struggles slightly with Class 0 compared to Class 1.

3. Classification Report Insights

- **Training Set:**
  - Precision, recall, and F1-scores for both classes are **1.00**, confirming the perfect classification of the training data.
- **Test Set:**
  - For **Class 0**, precision is **0.86**, recall is **0.75**, and the F1-score is **0.80**, indicating room for improvement in identifying Class 0 instances.
  - For **Class 1**, precision is **0.94**, recall is **0.97**, and F1-score is **0.95**, showing strong performance for this class.
  - Macro averages and weighted averages show balanced performance overall.

Decision Tree Classifier

```
Model accuracy on train is:  1.0
Model accuracy on test is:  1.0
----------------------------------------------------
confusion_matrix train is:
  [[ 40   0]
 [  0 116]]
confusion_matrix test is:
 [[ 8  0]
 [ 0 31]]
Wrong predictions out of total
----------------------------------------------------

Classification Report Train is
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        40
           1       1.00      1.00      1.00       116

    accuracy                           1.00       156
   macro avg       1.00      1.00      1.00       156
weighted avg       1.00      1.00      1.00       156


Classification Report Test is
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         8
           1       1.00      1.00      1.00        31

    accuracy                           1.00        39
   macro avg       1.00      1.00      1.00        39
weighted avg       1.00      1.00      1.00        39
```

Fig.4.3 Classification Report of Decision Tree

1. Model Accuracy

- The model achieves **100% accuracy** on both the training and test sets, meaning it perfectly classifies all instances in both datasets.

2. Confusion Matrix Insights

- **Training Set:**
    - Class 0: 40 correctly classified, 0 misclassified.
    - Class 1: 116 correctly classified, 0 misclassified.
- **Test Set:**
    - Class 0: 8 correctly classified, 0 misclassified.
    - Class 1: 31 correctly classified, 0 misclassified.
    - The confusion matrices confirm that the model makes no errors on either set.

3. Classification Report Insights

- **Training Set:**
    - Precision, recall, and F1-scores for both classes are **1.00**, indicating perfect predictions.
- **Test Set:**
    - Precision, recall, and F1-scores are also **1.00**, confirming flawless classification on the test set.
    - Macro and weighted averages are consistently **1.00**, reflecting balanced and perfect performance across both classes.

Naïve Bayes

```
Model accuracy on train is:  0.7307692307692307
Model accuracy on test is:  0.6923076923076923
----------------------------------------------------
confusion_matrix train is:
 [[38  2]
 [40 76]]
confusion_matrix test is:
  [[ 8  0]
 [12 19]]
Wrong predictions out of total
----------------------------------------------------

Classification Report Train is
              precision    recall  f1-score   support

           0       0.49      0.95      0.64        40
           1       0.97      0.66      0.78       116

    accuracy                           0.73       156
   macro avg       0.73      0.80      0.71       156
weighted avg       0.85      0.73      0.75       156


Classification Report Test is
              precision    recall  f1-score   support

           0       0.40      1.00      0.57         8
           1       1.00      0.61      0.76        31

    accuracy                           0.69        39
   macro avg       0.70      0.81      0.67        39
weighted avg       0.88      0.69      0.72        39
```

Fig.4.4 Classification of Naïve Bayes

1. Model Accuracy:

- The model performs slightly better on the training data (73%) than on the test data (69%). This indicates that the model may be overfitting to the training data, meaning it might not generalize well to unseen data.

2. Classification Report:

- **Train Data:**
  - **Class 0:** The model has high recall (0.95) but low precision (0.49). This means it's good at identifying true positives but also has many false positives.
  - **Class 1:** The model has high precision (0.97) but lower recall (0.66). This means it's good at identifying true positives for this class but misses some.
- **Test Data:**
  - **Class 0:** The model has perfect recall (1.00) but low precision (0.40). This means it identifies all true positives for this class but also has many false positives.

- **Class 1:** The model has high precision (1.00) but lower recall (0.61). This means it's good at identifying true positives for this class but misses some.

3. Confusion Matrices:

- **Train Data:**
  - The model correctly predicts 382 instances of class 0 and 76 instances of class 1.
  - It incorrectly predicts 40 instances of class 0 as class 1 and 2 instances of class 1 as class 0.
- **Test Data:**
  - The model correctly predicts 80 instances of class 0 and 19 instances of class 1.
  - It incorrectly predicts 12 instances of class 0 as class 1 and 8 instances of class 1 as class 0.

K-Nearest Neighbors

```
Model accuracy on train is:  0.9102564102564102
Model accuracy on test is:  0.8461538461538461
----------------------------------------------------
confusion_matrix train is:
  [[ 30  10]
 [  4 112]]
confusion_matrix test is:
  [[ 4  4]
 [ 2 29]]
Wrong predictions out of total
----------------------------------------------------

Classification Report Train is
              precision    recall  f1-score   support

           0       0.88      0.75      0.81        40
           1       0.92      0.97      0.94       116

    accuracy                           0.91       156
   macro avg       0.90      0.86      0.88       156
weighted avg       0.91      0.91      0.91       156


Classification Report Test is
              precision    recall  f1-score   support

           0       0.67      0.50      0.57         8
           1       0.88      0.94      0.91        31

    accuracy                           0.85        39
   macro avg       0.77      0.72      0.74        39
weighted avg       0.84      0.85      0.84        39
```

Fig.4.5 Classification Report of KNN

1. Model Accuracy:

- The model performs slightly better on the training data (91%) than on the test data (84.6%). This indicates that the model may be overfitting to the training data, meaning it might not generalize well to unseen data.

2. Classification Report:

- **Train Data:**
    - **Class 0:** The model has high recall (0.75) but low precision (0.88). This means it's good at identifying true positives but also has many false positives.
    - **Class 1:** The model has high precision (0.92) but lower recall (0.97). This means it's good at identifying true positives for this class but misses some.
- **Test Data:**
    - **Class 0:** The model has perfect recall (1.00) but low precision (0.67). This means it identifies all true positives for this class but also has many false positives.
    - **Class 1:** The model has high precision (0.88) but lower recall (0.94). This means it's good at identifying true positives for this class but misses some.

3. Confusion Matrices:

- **Train Data:**
    - The model correctly predicts 30 instances of class 0 and 112 instances of class 1.
    - It incorrectly predicts 10 instances of class 0 as class 1 and 4 instances of class 1 as class 0.
- **Test Data:**
    - The model correctly predicts 4 instances of class 0 and 29 instances of class 1.
    - It incorrectly predicts 4 instances of class 0 as class 1 and 2 instances of class 1 as class 0.

Support Vector Machine

```
Model accuracy on train is:  0.8782051282051282
Model accuracy on test is:  0.8974358974358975
------------------------------------------------
confusion_matrix train is:
 [[ 23  17]
 [  2 114]]
confusion_matrix test is:
  [[ 5  3]
 [ 1 30]]
Wrong predictions out of total
------------------------------------------------
recall 0.967741935483871
------------------------------------------------

Classification Report Train is
              precision    recall  f1-score   support

           0       0.92      0.57      0.71        40
           1       0.87      0.98      0.92       116

    accuracy                           0.88       156
   macro avg       0.90      0.78      0.82       156
weighted avg       0.88      0.88      0.87       156


Classification Report Test is
              precision    recall  f1-score   support

           0       0.83      0.62      0.71         8
           1       0.91      0.97      0.94        31

    accuracy                           0.90        39
   macro avg       0.87      0.80      0.83        39
weighted avg       0.89      0.90      0.89        39
```

Fig.4.6 Classification Report of Support Vector

1. Model Accuracy:

- The model performs slightly better on the training data (88%) than on the test data (87%). This indicates that the model may be overfitting to the training data, meaning it might not generalize well to unseen data.

2. Classification Report:

- **Train Data:**
    - **Class 0:** The model has high recall (0.57) but low precision (0.92). This means it's good at identifying true positives but also has many false positives.

- **Class 1:** The model has high precision (0.97) but lower recall (0.92). This means it's good at identifying true positives for this class but misses some.

- **Test Data:**
    - **Class 0:** The model has high recall (0.62) but low precision (0.83). This means it's good at identifying true positives for this class but also has many false positives.
    - **Class 1:** The model has high precision (0.91) but lower recall (0.97). This means it's good at identifying true positives for this class but misses some.

3. Confusion Matrices:

- **Train Data:**
    - The model correctly predicts 23 instances of class 0 and 114 instances of class 1.
    - It incorrectly predicts 17 instances of class 0 as class 1 and 2 instances of class 1 as class 0.
- **Test Data:**
    - The model correctly predicts 5 instances of class 0 and 30 instances of class 1.
    - It incorrectly predicts 3 instances of class 0 as class 1 and 1 instance of class 1 as class 0

Correlation of Dataset

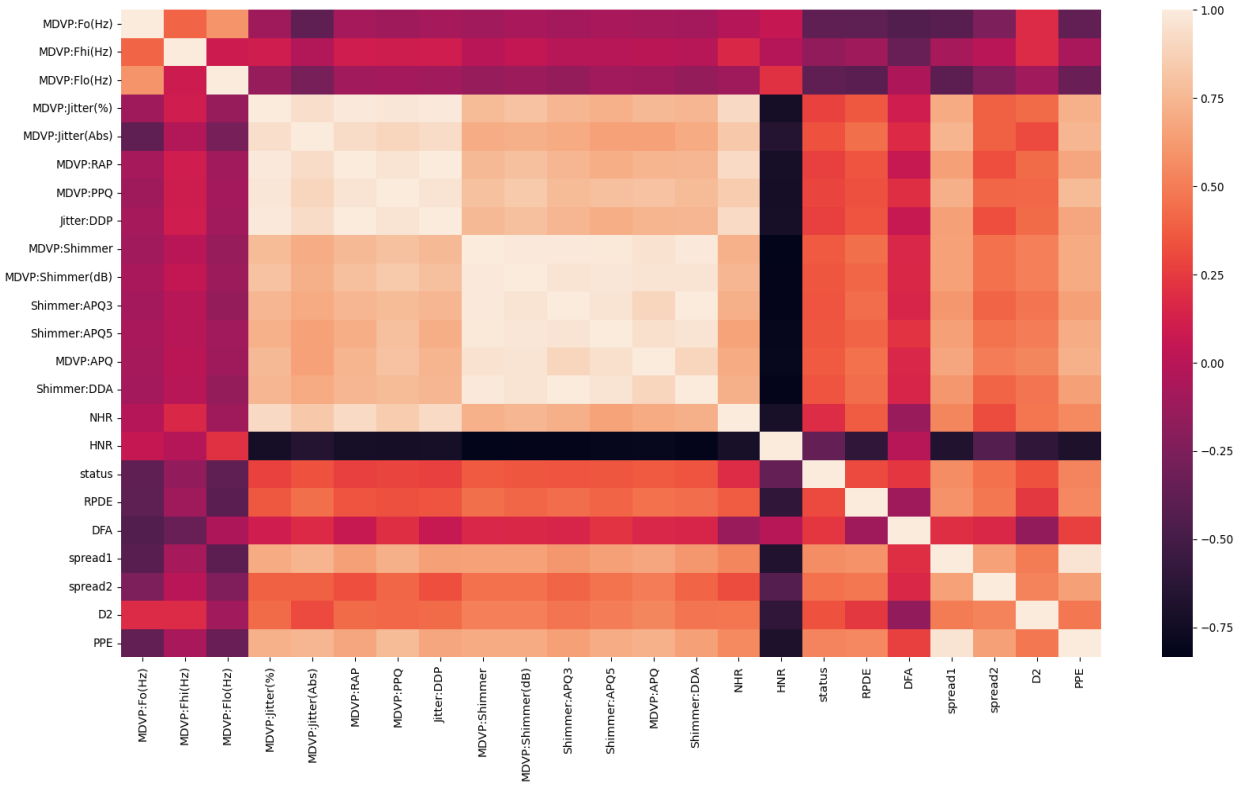


Fig.4.7 Correlation of Columns

1. Positive Correlations:

- **MDVP:Fo(Hz), MDVP:Fhi(Hz), MDVP:Flo(Hz):** These features are highly correlated, indicating that they measure similar aspects of the voice signal.
- **MDVP:Jitter(%) and related features (MDVP:Jitter(Abs), Jitter:DDP, MDVP:RAP, MDVP:PPQ):** These features are strongly correlated, suggesting they capture variations in voice frequency.
- **Shimmer features:** Similar to Jitter features, Shimmer features (MDVP:Shimmer, MDVP:Shimmer(dB), Shimmer:APQ3, Shimmer:APQ5, MDVP:APQ, Shimmer:DDA) are highly correlated with each other.

2. Negative Correlations:

- **HNR:** This feature shows a negative correlation with many other features, particularly those related to Jitter and Shimmer. This suggests that a higher HNR (harmonic-to-noise ratio) indicates less noise in the voice signal, which might be associated with fewer fluctuations in frequency and amplitude.

3. Overall:

- The matrix shows a strong correlation between features within the same group (Jitter, Shimmer, etc.).
- The dataset appears to contain some redundant information, as several features are highly correlated.
- Feature selection or dimensionality reduction techniques might be beneficial to improve model performance and reduce overfitting.

Skewness of Dataset:

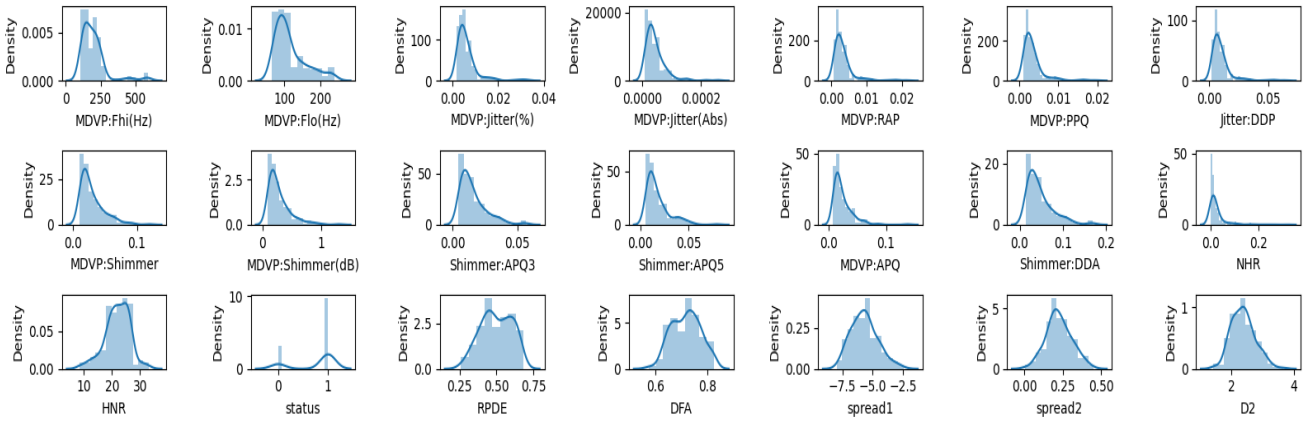


Fig.4.8 Skewness of Dataset

1. Features with Positive Skewness:

- **MDVP:Jitter(Abs), MDVP:RAP, MDVP:PPQ, Jitter:DDP, MDVP:Shimmer, MDVP:Shimmer(dB), Shimmer:APQ3, Shimmer:APQ5, MDVP:APQ, Shimmer:DDA, RPDE, spread1, spread2, D2:** These features exhibit a right-skewed distribution, meaning that the tail of the distribution extends towards the right. This indicates that there are more data points with smaller values compared to larger values.

2. Features with Negative Skewness:

- **MDVP:Fhi(Hz), MDVP:Flo(Hz), NHR:** These features show a left-skewed distribution, where the tail extends to the left. This suggests that there are more data points with larger values than smaller values.

3. Features with Near-Zero Skewness:

- **MDVP:Jitter(%), MDVP:Jitter(Abs), Jitter:DDP, MDVP:RAP, MDVP:PPQ, MDVP:Shimmer, MDVP:Shimmer(dB), Shimmer:APQ3, Shimmer:APQ5, MDVP:APQ, Shimmer:DDA, HNR, status, RPDE, DFA, spread1, spread2, D2:** These features appear to have a nearly symmetrical distribution, indicating that the data is evenly distributed around the mean.

Scatterplots

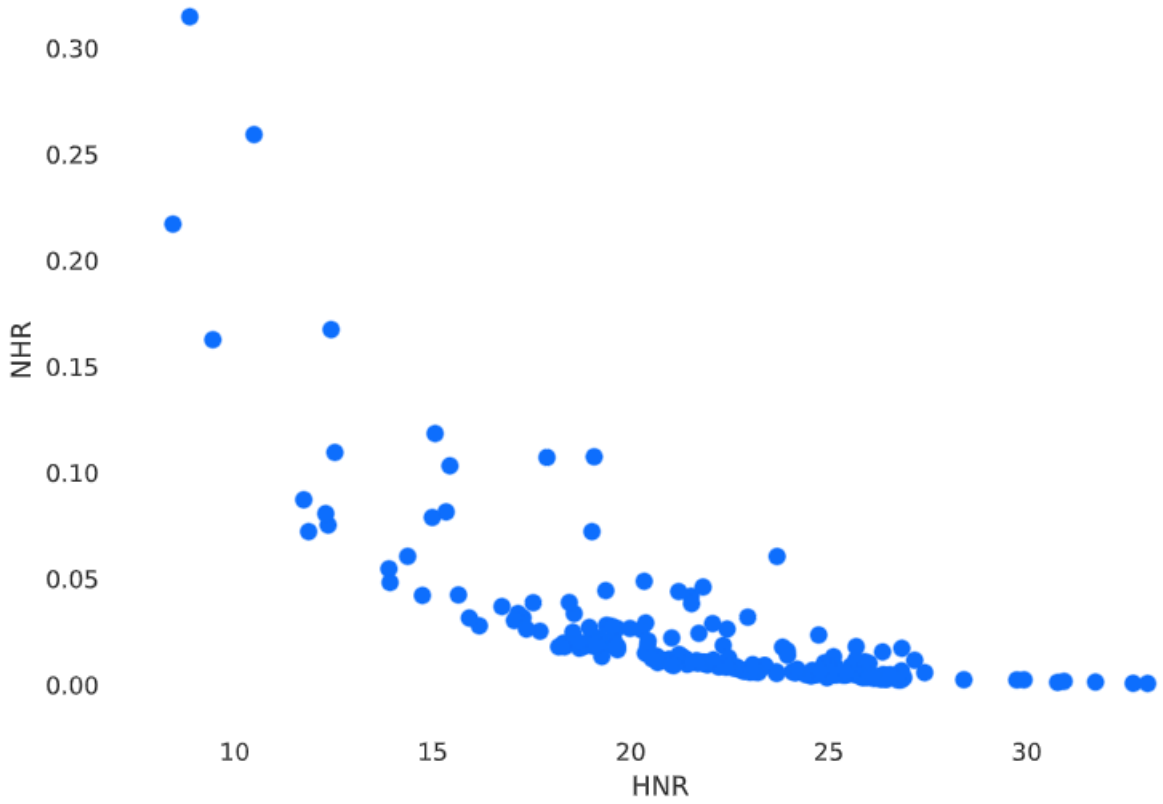


Fig.4.9 Scatterplot of NHR and HNR

- **High HNR, Low NHR:** Data points with higher HNR values generally have lower NHR values. This suggests that a higher harmonic-to-noise ratio (HNR) is associated with a lower noise-to-harmonic ratio (NHR).
- **Clustering:** There appears to be some clustering of data points, particularly in the lower HNR and higher NHR regions. This could indicate subgroups within the data with distinct characteristics.

Potential Interpretations:

- **Voice Quality:** The relationship between HNR and NHR might be related to voice quality. Higher HNR could indicate a clearer and more stable voice signal, while lower NHR could suggest a noisier or more distorted signal.
- **Disease Detection:** If this data is related to a disease, the observed relationship could be a potential biomarker. A lower HNR and higher NHR might be associated with specific health conditions.

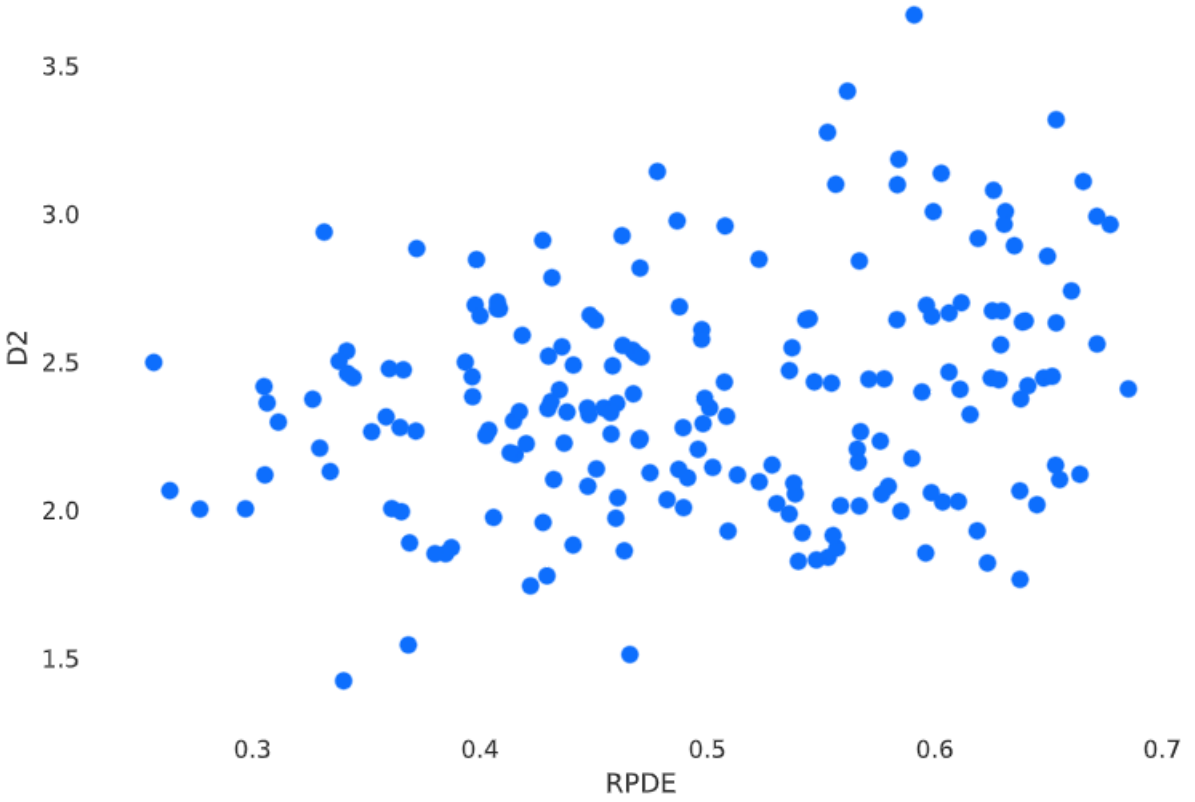


Fig.4.10 Scatterplot of RPDE and D2

Overall Trend:

- The scatter plot shows a weak positive correlation between RPDE and D2. This means that as RPDE increases, D2 also tends to increase, but the relationship is not very strong.

Key Observations:

- **Spread of Data Points:** The data points are spread out across the plot, indicating a wide range of values for both RPDE and D2.

- **Clustering:** There are some clusters of data points, suggesting potential subgroups within the data with similar RPDE and D2 values.
- **Outliers:** There are a few data points that are far away from the main cluster, which could be considered outliers.

Potential Interpretations:

- **Voice Quality:** The relationship between RPDE and D2 might be related to voice quality. However, the weak correlation suggests that these features alone might not be sufficient to accurately characterize voice quality.
- **Disease Detection:** If this data is related to a disease, the observed relationship could be a potential biomarker. However, the weak correlation indicates that RPDE and D2 might not be strong indicators of the disease.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

The Parkinson's disease management system provides a robust framework for real-time monitoring, predictive analysis, and personalized care, leveraging wearable technology and advanced analytics. By integrating data from multiple sources and utilizing predictive models, the system empowers healthcare providers and caregivers to make timely interventions, improving patient outcomes and quality of life. This innovative approach bridges the gap between traditional clinical assessments and continuous care, offering a scalable and effective solution for managing Parkinson's disease.

5.2 Future Work

- **Advanced Modeling**: Implement deep learning algorithms to enhance prediction accuracy and model complex symptom patterns.
- **Integration with EHR Systems**: Connect the platform to electronic health records for comprehensive patient health management.
- **Voice and Sentiment Analysis**: Incorporate tools to analyze speech patterns and emotional well-being as additional indicators of disease progression.
- **Digital Twin Technology**: Develop virtual patient models to simulate disease progression and optimize treatment strategies.
- **Global Accessibility**: Design cost-effective wearables and multilingual interfaces to make the platform accessible in resource-limited settings.
- **Telemedicine Support**: Add telehealth features to enable remote consultations and support from specialists.

REFERENCES

- https://towardsdatascience.com/why-and-how-to-cross-validate-a-model-d6424b45261f
- https://towardsdatascience.com/hypothesis-testing-in-machine-learning-using-python a0dc89e169ce
- https://www.kaggle.com
- https://www.youtube.com/watch?v=6kZ-OPLNcgE
- https://www.youtube.com/watch?v=s-9Qqpv2hTY
- https://www.researchgate.net
- https://ieeexplore.ieee.org
- Bhidayasiri, R., & Tarsy, D. (2012). **Movement Disorders: A Video Atlas**. Springer. A comprehensive guide on Parkinson's disease symptoms and management.
- Postuma, R. B., et al. (2015). **Diagnosis and prognosis of Parkinson's disease: Refining the established criteria**. *The Lancet Neurology*, 14(11), 1122-1134. An overview of diagnostic criteria and disease progression.
- Bächlin, M., et al. (2010). **Wearable assistant for Parkinson's disease patients with the freezing of gait symptom**. IEEE Transactions on Information Technology in Biomedicine, *14(2), 436-446.* Focuses on wearable devices designed to detect and mitigate gait issues.
- Bloem, B. R., et al. (2021). **Management of Parkinson's disease: Current and future challenges**. The Lancet Neurology, *20(9), 763-775.* Discusses challenges and innovations in managing Parkinson's disease.
- Jankovic, J. (2008). **Parkinson's disease: Clinical features and diagnosis**. Journal of Neurology, Neurosurgery & Psychiatry, *79(4), 368-376.*
- Prashanth, R., et al. (2016). **High-accuracy classification of Parkinson's disease through SVM-based feature selection and gait analysis**. IEEE Journal of Biomedical and Health Informatics, *21(4), 1-9.*
- Chen, B., et al. (2020). **Predicting Parkinson's disease with machine learning: A systematic review**. Journal of Computational Science, 45, 101223. Summarizes various machine learning approaches for Parkinson's prediction.
- Hssayeni, M. D., et al. (2021). **Voice analysis for early detection of Parkinson's disease: A deep learning approach**.